

Delegates and Events

Delegates

- delegate variables store references to other methods
- the references can be added or removed at runtime
- the referenced methods must have the same signature (same structure) as the delegate type

```
public delegate string MyDelegate(string par1, int par2);

public string Introduction(string name, int age)
{
    return $"Hello my name is {name} and I am {age} years old.";
}

MyDelegate del = new MyDelegate(Introduction);
del("Max", 32);
//Hello my name is Max and I am 32 years old.
```

Delegates

- Each delegate variable can reference any number of methods, which are executed one after the other
- Methods can be added by += and detached by -=

```
public delegate string MyDelegate(string par1, int par2);

public string IntroductionDE(string name, int alter)
{
    return $"Hallo mein Name ist {name} und ich bin {alter} Jahre alt.";
}

public string IntroductionEN(string name, int age)
{
    return $"Hello my name is {name} and I am {age} years old.";
}
```

```
MyDelegate del = new MyDelegate(IntroductionDE);
del += IntroductionEN;
del("Max", 32);
//Hallo mein Name ist Max und ich bin 32 Jahre alt.
//Hello my name is Max and I am 32 years old.
```

predefined Delegate types

Name	Return value	Parameter	Example
Action Action<in T1> Action<in T1, in T2> ...	void	0-16	<pre>public void PrintInteger(int p1) { Console.Write(p1); }</pre> Action<int> a = PrintInteger
Predicate<in T>	bool	1	<pre>public bool CheckForZero(int p1) { return p1 == 0; }</pre> Predicate<int> p = CheckForZero
Func<out TResult> Func<in T1, out TResult> Func<in T1, in T2, out TResult> ...	TResult	0-16	<pre>public string SumToString(int p1, int p2) { return (p1 + p2).ToString(); }</pre> Func<int, int, string> f = SumToString

anonyme Methoden

- Methods without names are called anonymous methods
- Methods can be assigned to a delegate without declaring them first

```
Func<int, int> meineFunc = delegate (int param1)
{
    return param1 + 5;
};
```

- 'Disposable' methods for one-time use, e.g. to pass additional logic to methods.

```
List<string> StädteListe = new List<string>() { "München", "Berlin", "Hamburg", "Hannover", "Köln" };
string gefundeneStadt = StädteListe.Find
(
    delegate (string stadt)
    {
        return stadt.StartsWith('B');
    }
);
```

Lambda - Expressions

- Shorthand notation for anonymous methods

```
Func<int, int, int> Add = delegate(int x, int y) {return x+y;}
```



```
Func<int, int , int> Add = (int x, int y) => {return x+y;}
```



```
Func<int, int , int> Add = (x, y) => x+y;
```

EventHandler

- An event is a delegate variable to which any number of methods (EventListener) can log on and off.
- delegate type is usually EventHandler
- Expected parameters: object, EventArgs
- No 'overwriting' by new assignment possible

```
event EventHandler onClick;
```

```
onClick += delegate (object sender, EventArgs EventArgs)  
{  
    Console.WriteLine("EventBehandlung");  
};
```

```
onClick(this, EventArgs.Empty);
```